

## Graph - Data Structure

- Basics
- BFS & applications
- DFS & applications

## Basic Definitions

A graph  $G$  is a pair of sets  $(V, E)$ , where  $V$  is an arbitrary non-empty set and  $E$  is a set of pairs of elements of  $V$ .

Elements of  $V$  and  $E$  are called vertices (nodes) and edges (links) of  $G$  respectively.

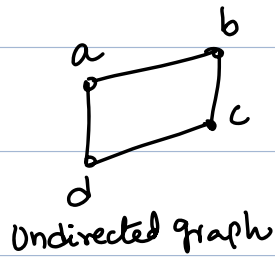
In an undirected graph, the edges are unordered pairs (sets of size two). We use  $uv$  instead of  $\{u, v\}$  to denote the undirected edge between vertices  $u$  and  $v$ .

In a directed graph, the edges are ordered pairs of vertices. We use  $(u, v)$  to denote the directed edge from  $u$  to  $v$ .

Usually  $n$  - denotes # of vertices.

$m$  - " " " edges.

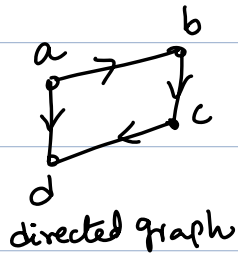
Eg ①



$$V = \{a, b, c, d\}$$

$$E = \{ \{a, b\}, \{b, c\}, \{c, d\}, \{d, a\} \}$$

Eg ②



$$V = \{a, b, c, d\}$$

$$E = \{ (a, b), (b, c), (c, d), (d, a) \}$$

Real life examples: Road networks, Social networks, etc

let  $G=(V,E)$  be an undirected graph.

Path: A path  $P$  is a sequence of distinct vertices

$v_1, v_2, \dots, v_k$  such that each consecutive pair

$v_i, v_{i+1}$  is joined by an edge in  $G$ .

$P$  is called a path from  $v_1$  to  $v_k$ .

Cycle: A cycle is a path  $v_1, v_2, \dots, v_k$ , ( $k \geq 2$ )

with  $v_1 = v_k$ .

Connected: An undirected graph is connected if

for every pair of nodes  $u$  and  $v$  there is a

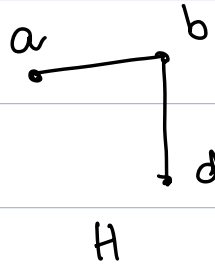
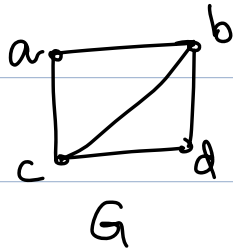
path from  $u$  to  $v$ .

Tree: An undirected graph is a tree if it is

connected and does not contain a cycle.

Subgraph: A Subgraph of a graph  $G$  is another graph formed from a subset of the vertices and edges of  $G$ .

Example:



$H$  is a Subgraph of  $G$ .

Exercise

- Prove that
- ① Every tree on  $n$  vertices has exactly  $n-1$  edges

## Other Terminology

① adjacent

② degree

③ Subgraph, induced subgraph

④ Walk, Path, Cycle, length of the path/cycle

⑤ Connected graph, Components, Strongly Connected

⑥ Tree, Forest, DAG

⑦ distance in the graphs (weighted vs unweighted)

## Representation of Graphs

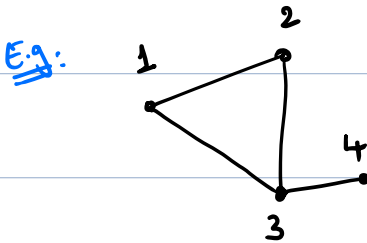
There are several ways to represent graphs, each with its advantages and disadvantages.

In this course we will see three ways to represent graphs.

- ① Edge lists
- ② Adjacency matrices
- ③ Adjacency Lists

## Edge list

An edge list is a list or array of all the edges of the graph.



Array = [ [1 2] [2 3] [1 3] [3 4] ]

Total space for an edge list is  $O(E)$

### disadvantage:

If we want to find whether the graph contains a particular edge, we have to search through the edge list, In the worst case we have to search through  $|E|$  edges.



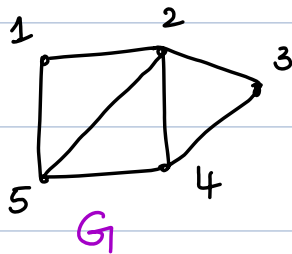
## Adjacency matrices

We assume that the vertices are numbered  $1, 2, \dots, |V|$  in some arbitrary manner.

The adjacency matrix of a graph  $G$  is a  $V \times V$  matrix  $A = (a_{ij})$  of 0's and 1's such that

$$a_{ij} = \begin{cases} 1 & \text{if } ij \in E \\ 0 & \text{otherwise} \end{cases}$$

Example:



$A =$

$$\begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

Adjacency matrix representation of  $G$ .

Remark: For an undirected graph, the adjacency matrix is symmetric.

For a directed graph the adjacency matrix need not be symmetric.

We can find out whether an edge is present in constant time by just looking up the corresponding entry in the matrix

### disadvantage :

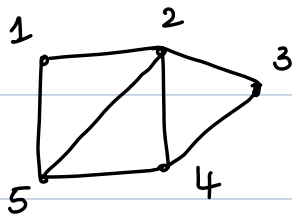
① Adjacency matrix representation of a graph requires  $\Theta(n^2)$  space, irrespective of the # of edges in the graph.

② Finding which vertices are adjacent to a given vertex  $i$ , we have to look at all  $|V|$  entries in row  $i$ , even if only a small number of vertices are adjacent to a vertex  $i$

## Adjacency Lists :

An adjacency list is an array of lists,  
each containing the neighbors of one of the vertices  
(out-neighbors if the graph is directed).

Example:



1 [ ] → [2] → [5] /

2 [ ] → [1] → [5] → [3] → [4] /

3 [ ] → [2] → [4] /

4 [ ] → [2] → [5] → [3] /

5 [ ] → [4] → [1] → [2] /

This representation uses an array indexed by vertex number, in which the array cell for each vertex points to a singly linked list of the neighboring vertices.

① The overall space required for an adjacency list is  $O(n+m)$  { some books write  $O(V+E)$  }  
↓                      ↓  
# of vertices      # of edges  
 $O(|V|+|E|)$

② The standard way to implement the adjacency list is using an array (list) of single-linked lists, where the head of the linked list is the vertex and all the connected linked lists are the vertices to which it is connected.

③ we can find neighbors of a vertex  $i$  in  $\Theta(d_i)$  time, where  $d_i$  is the degree of vertex  $i$ .

## Comparison :

|                                                           | Adjacency list                                                     | Adjacency matrix                               |
|-----------------------------------------------------------|--------------------------------------------------------------------|------------------------------------------------|
| ① Space                                                   | $\Theta(V + E)$                                                    | $\Theta(V^2)$                                  |
| ② Test if $UV \in E$                                      | $O(1 + \min\{\deg(u), \deg(v)\})$<br>$\parallel O(V)$ (worst case) | $O(1)$                                         |
| ③ Test if $(u, v) \in E$<br>$\downarrow$<br>directed edge | <u><math>O(1 + \deg(u)) = O(V)</math></u>                          | <u><math>O(1)</math></u>                       |
| ④ List $v$ 's (out)-neighbors                             | $\Theta(1 + \deg(v)) = O(V)$                                       | $\Theta(V)$                                    |
| ⑤ List all edges                                          | $\Theta(V + E)$                                                    | $\Theta(V^2)$                                  |
| ⑥ Insert edge $uv$                                        | $O(1)$                                                             | $O(1)$                                         |
| ⑦ Delete edge $uv$                                        | $O(\deg(u) + \deg(v)) = O(V)$                                      | $O(1)$                                         |
| ⑧ Adding a vertex                                         | $O(1)$                                                             | $O(V^2)$<br>(Storage space must be increased.) |
| ⑨ Adding an edge                                          | $O(1)$                                                             | $O(1)$                                         |
| ⑩ Delete a vertex                                         | $O(V + E)$                                                         | $O(V^2)$                                       |

Relationship b/w  $n$  &  $m$ :

If  $G$  is <sup>undirected &</sup> Connected & without Parallel edges & self loops

then

$$n-1 \leq m \leq \binom{n}{2}$$

Sparse vs Dense Graphs

In most applications  $m$  is  $\Omega(n)$  and  $O(n^2)$

(Roughly)

- In a sparse graph  $m$  is  $O(n)$  or close to it
- In a dense graph  $m$  is close to  $\Theta(n^2)$

## Adjacency Lists ( we use this representation in this course)

if a graph is sparse, then most of entries in adjacency matrix are zero (which is a waste of space). Hence, we generally use adjacency list representation for sparse graphs.

\* If  $G$  has  $n$  vertices &  $m$  edges then

Adjacency list representation requires  $\Theta(m+n)$  space.

Q: Which is better? adjacency matrix or adjacency list.

Ans: Depends on density of the graph and operations we need to form.